iSingLab HKUST

DeepEP
Expert-Parallel Communication

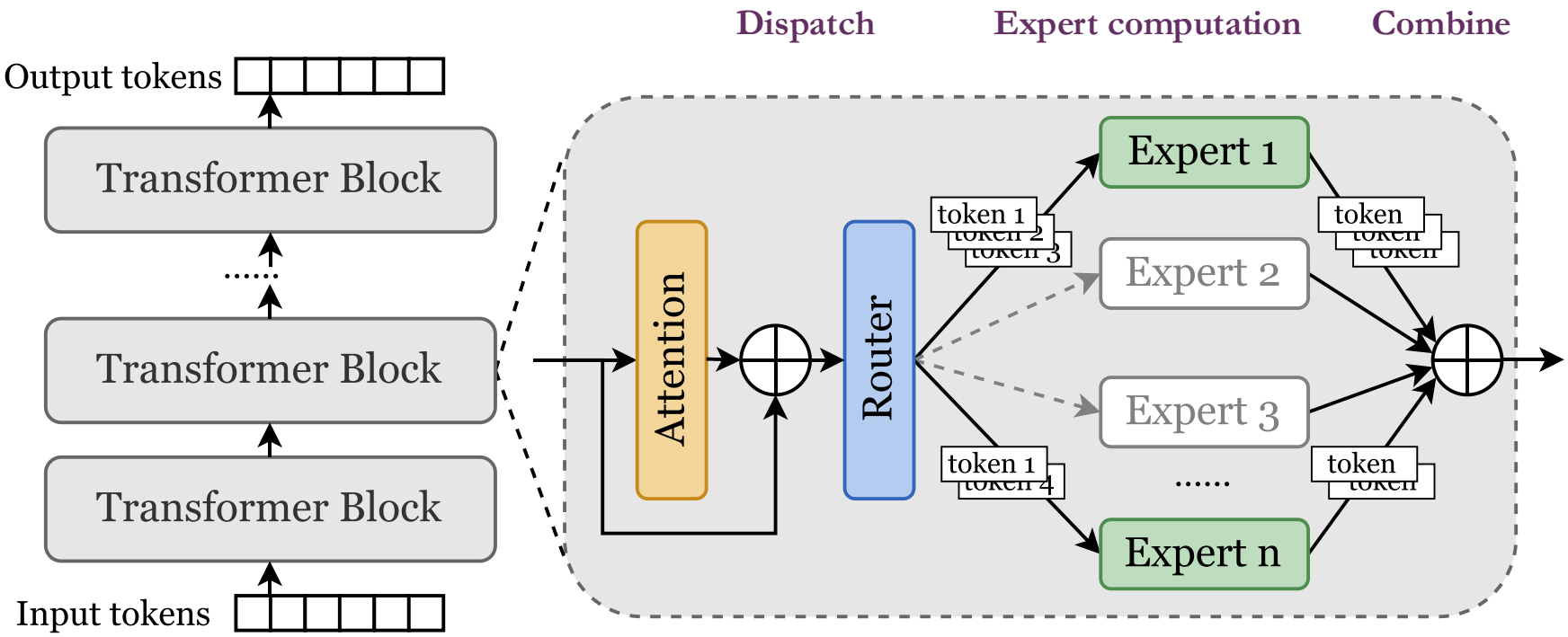
GPU-Initiated RDMA and NVLink Forwarding
Tutorial for Lecture 03

Advanced AI Computing and Systems

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MoE Communication

MoE Dispatch and Combine



· Dispatch moves each token to its selected experts before expert computation.
· Combine returns expert outputs and reduces them with routing weights.

MoE Communication

Expert IDs and GPU Ranks

```
rank = expert_id / 9        local_expert = expert_id % 9
```

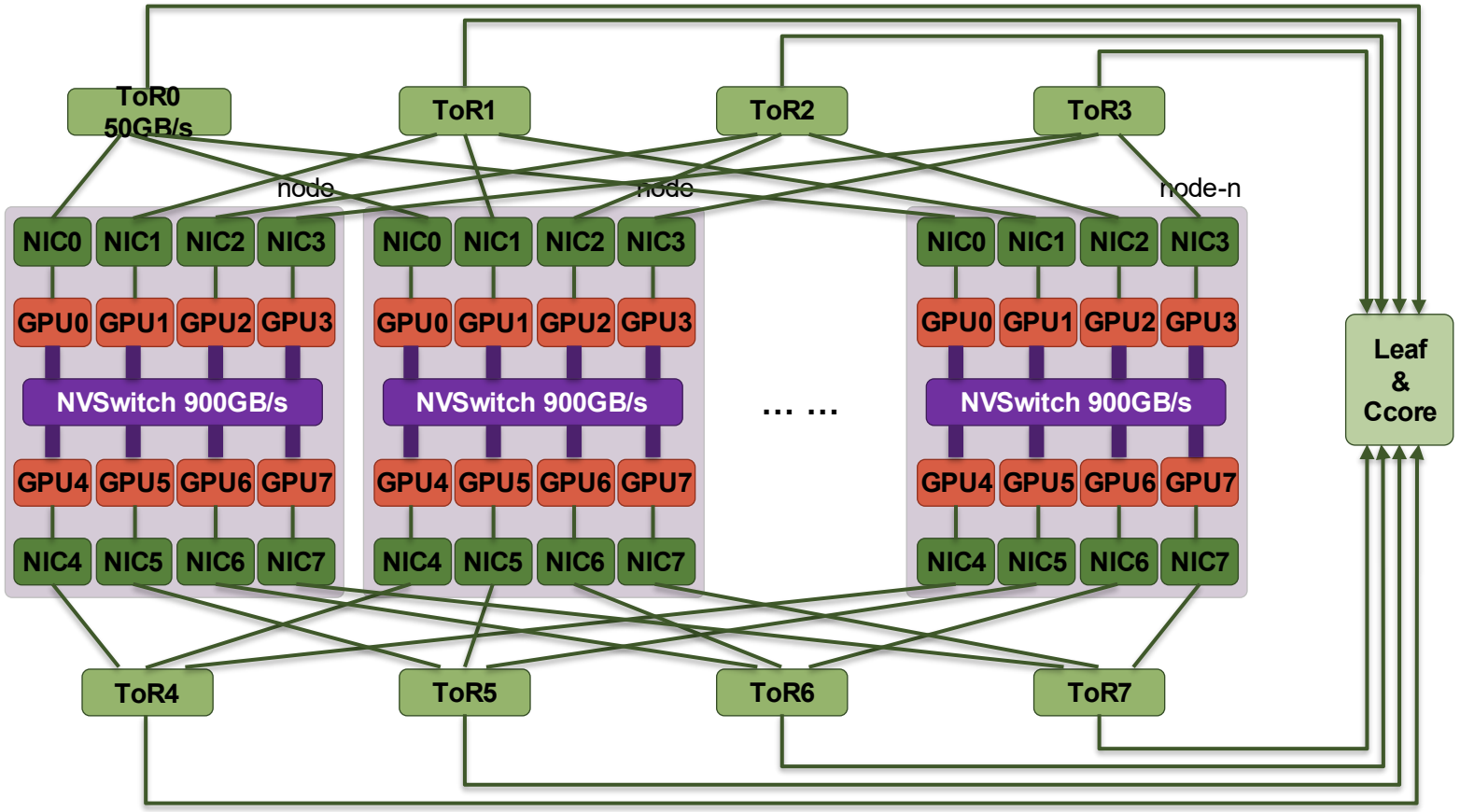
Global expert ID	Destination GPU rank	Local expert ID
0	0	0
8	0	8
9	1	0
287	31	8

DeepEP v1.2.1 · Reference R4, R10, R17

- Routing selects global expert IDs; an expert ID is not a GPU rank.
- Contiguous placement maps expert e to rank floor(e / L) and local expert e % L.

GPU Communication

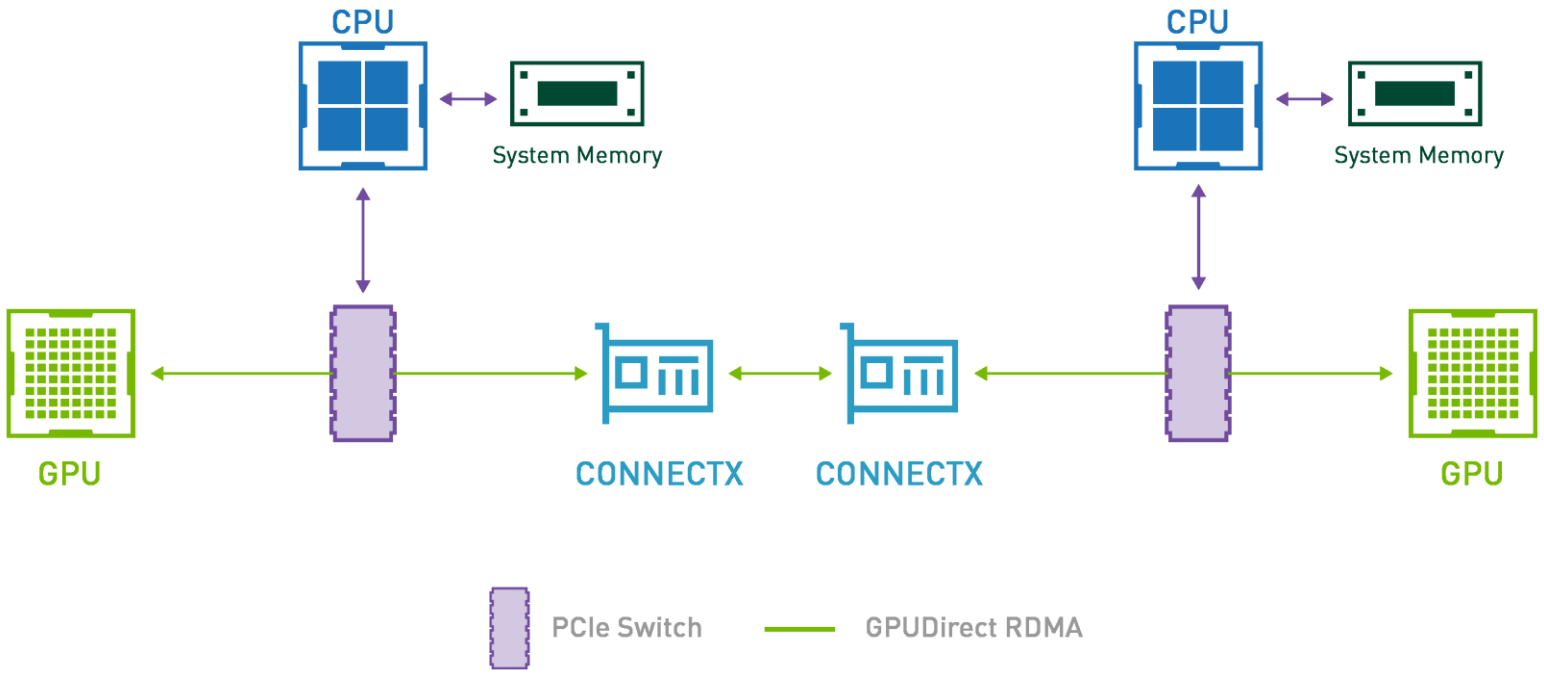
NVLink and RDMA Domains



- NVLink connects local GPU peers; RDMA connects GPU memory through NICs.
- The throughput path composes these domains instead of treating every hop alike.

GPU Communication

GPUDirect RDMA

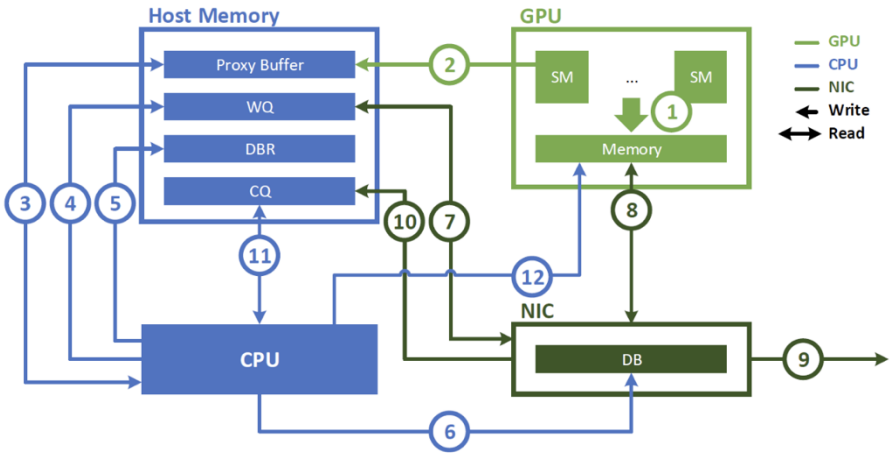


- The NIC reads or writes registered GPU memory without a host-memory staging copy.
- Direct data movement still requires setup, valid memory mappings, and synchronization.

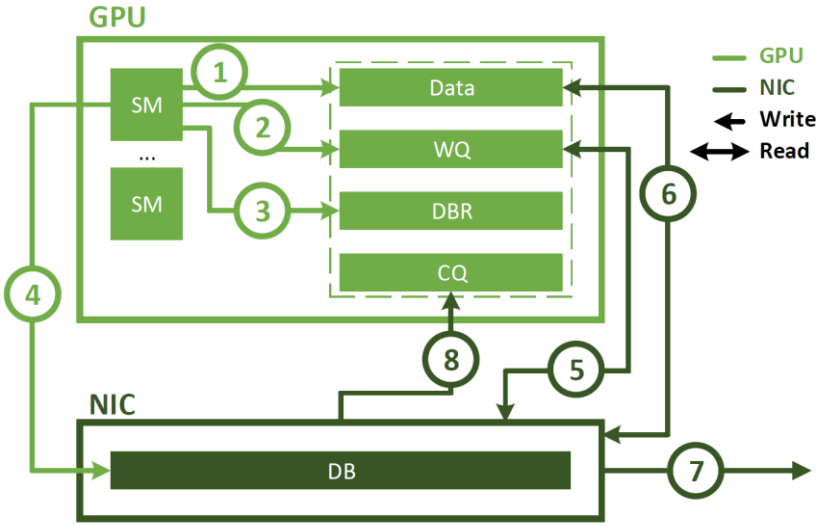
GPU Communication

GPU-Initiated RDMA with IBGDA

CPU proxy submission



GPU submission with IBGDA



DeepEP v1.2.1 · Reference R6, R18

· GPU threads prepare work queue entries and notify the NIC through a doorbell.
· CPU setup establishes resources; the GPU can then issue communication in the kernel.

DeepEP Overview

Two Dispatch Paths

Low-latency dispatch

This case: peer NVLink disabled

Source GPU → RDMA → Expert GPU

High-throughput dispatch

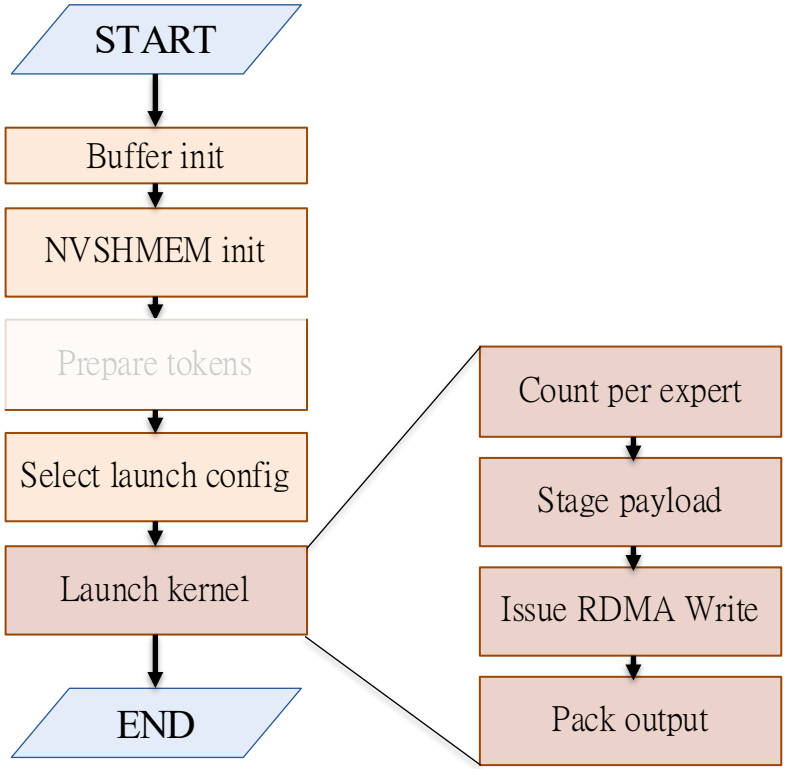
Source GPU → RDMA peer → NVLink → Expert GPU

Self-rank delivery can use a local-copy branch.

- Low latency: use fixed receive slots and publish counts after issuing token writes.
- High throughput: exchange layout information and pipeline RDMA with NVLink forwarding.

Low-Latency Dispatch

Dispatch Stages and Buffer Lifetimes



Before repeated dispatch

Create communication resources
and register reusable storage.

Each invocation

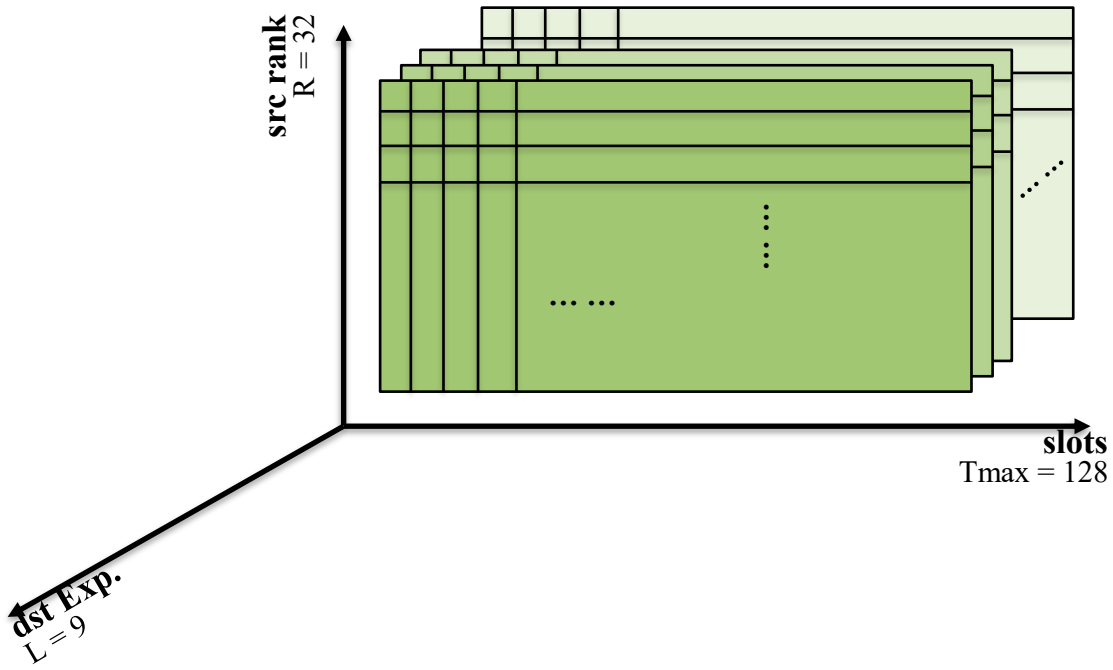
Prepare → submit → notify → receive and pack.

· Stage token data, issue writes, publish counts, then pack the received messages.
· A buffer stays live until its readers and the communication protocol have finished.

Low-Latency Dispatch

Fixed Receive Slots

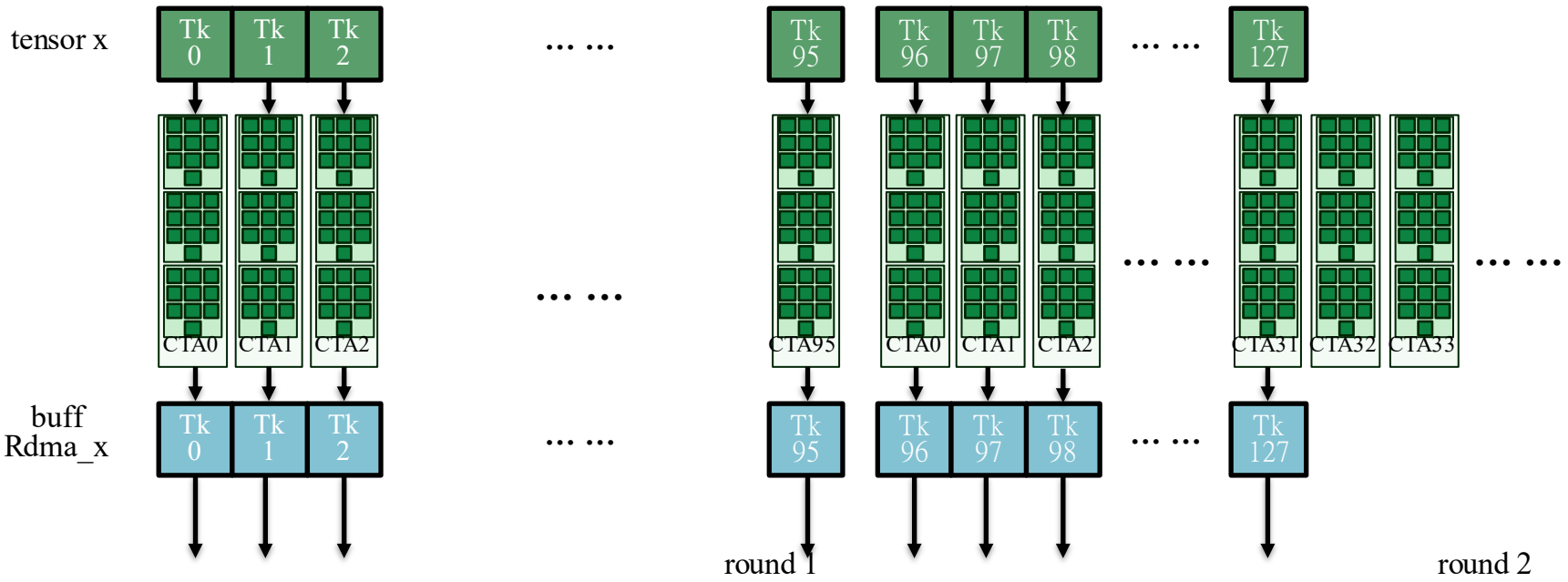
rdma_recv_x[local expert][source rank][slot]



- Reserve a separate receive slice for every local-expert and source-rank pair.
- Known addresses let payload writes start before a receiver-side count exchange.

Low-Latency Dispatch

Packing Tokens into Registered Buffers

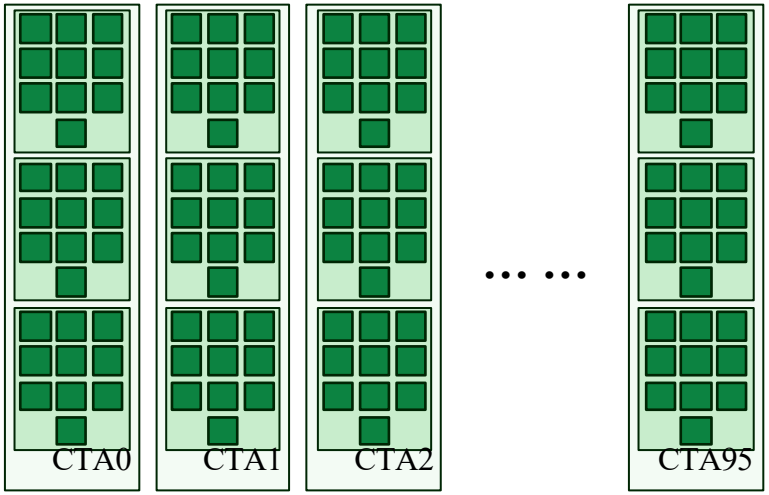


- Stage each token with source identity and, when enabled, FP8 scaling factors.
- The communication buffer gives the NIC registered addresses, keys, and stable storage.

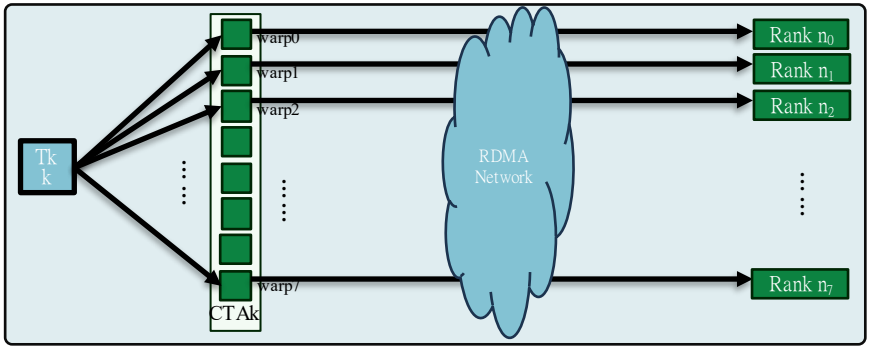
Low-Latency Dispatch

Mapping Token Sends to Warps

Launch blocks and software groups



One token, several expert targets



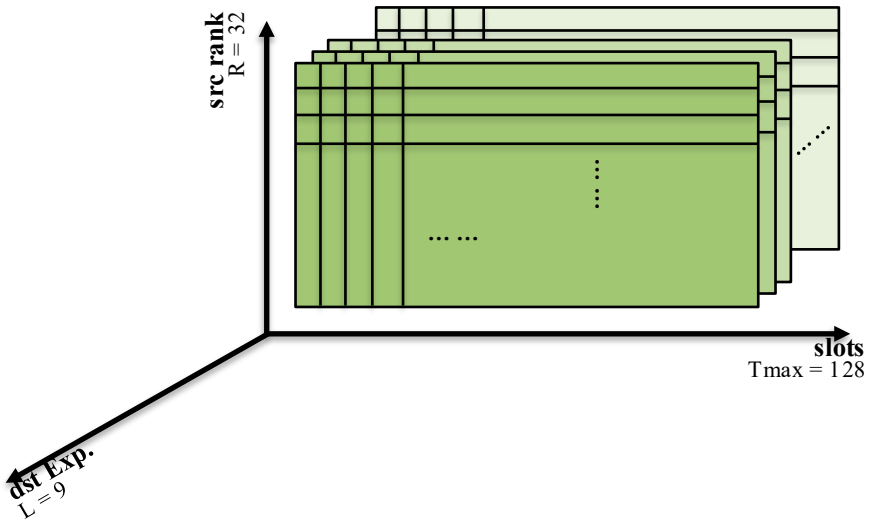
Example launch: 96 CTAs × 960 threads; scheduling determines physical SM placement.

- A CTA cooperatively stages a token; selected warps issue its expert-directed writes.
- A separate counting warp tracks the experts assigned to that CTA.

Low-Latency Dispatch

Reserving Slots with Atomics

A partition belongs to one source rank.




```
slot = atomicAdd(&reserve[e], 1);

peer = e / 9;
local_e = e % 9;

put_nbi(remote[local_e][rank][slot],
        staged_token, peer);
```

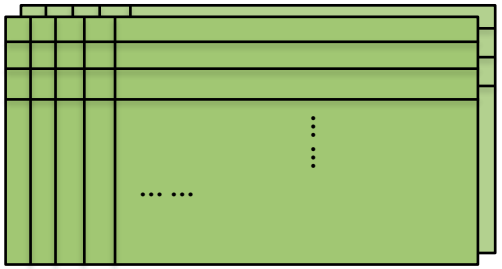
Each concurrent sender receives a different slot.

- One atomic increment allocates a unique slot among local sends to the same expert.
- Source-rank slices separate GPUs; the atomic separates concurrent warps on one GPU.

Low-Latency Dispatch

Publishing Readiness to the Receiver

Sender GPU



Ordered peer / QP

```
post payload WRITEs
publish local issued progress
wait until local requests are issued
post count AMO: -(n + 1)
```

n = 0 still publishes -1, so the receiver can finish waiting.

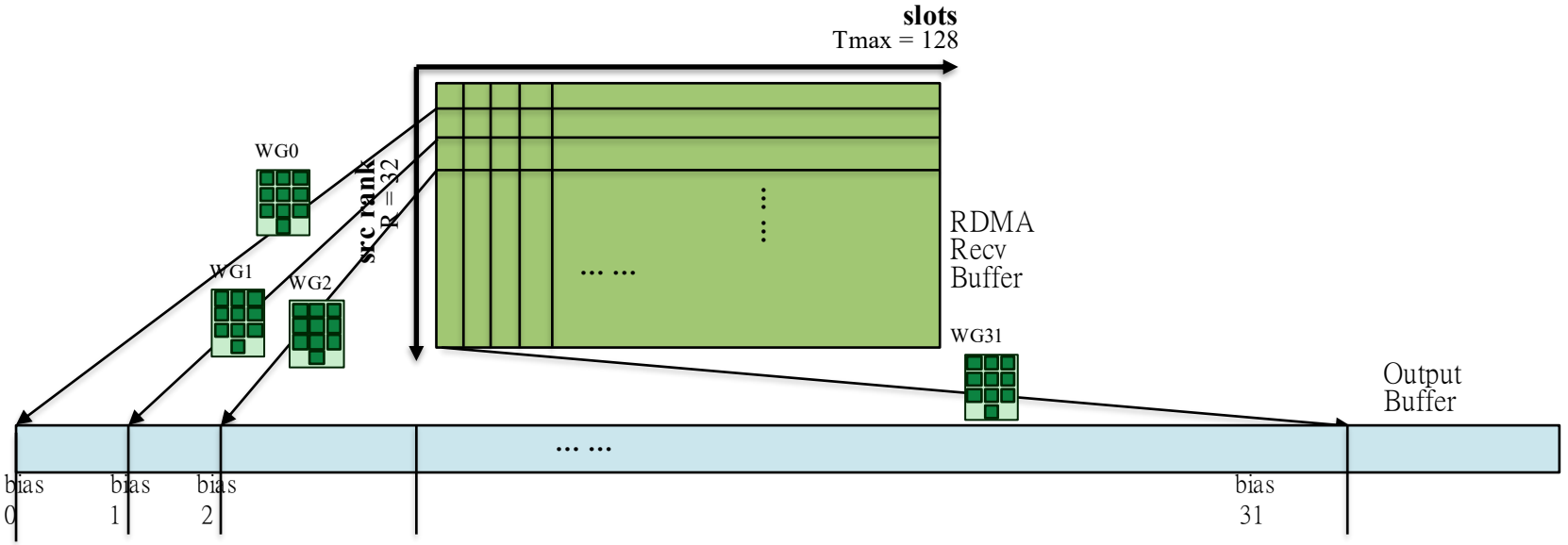
```
while ((flag = load_acquire(count)) == 0) {}
n = -flag - 1;
// Ordered notification makes the payload ready.
```

· The local finish counter records issued sends, not remote payload completion.
· A count AMO follows payload writes on the same QP; encode n tokens as -(n + 1).

Low-Latency Dispatch

Receiving and Packing Expert Inputs

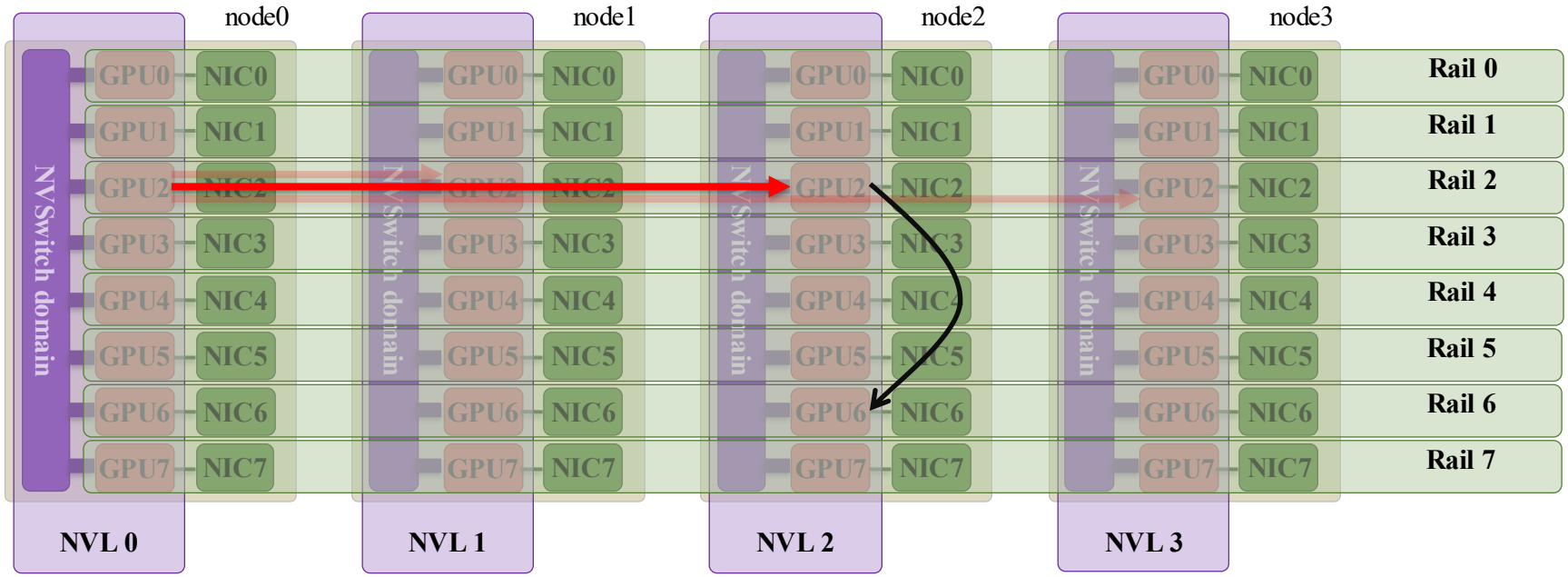
```
n = -ready_flag - 1;   begin = atomicAdd(&packed_count[local_e], n);
```



- Wait for a nonzero count flag, decode n, and reserve n contiguous output positions.
- Pack source slices into each expert's valid prefix while preserving return metadata.

High-Throughput Dispatch

RDMA-to-NVLink Forwarding

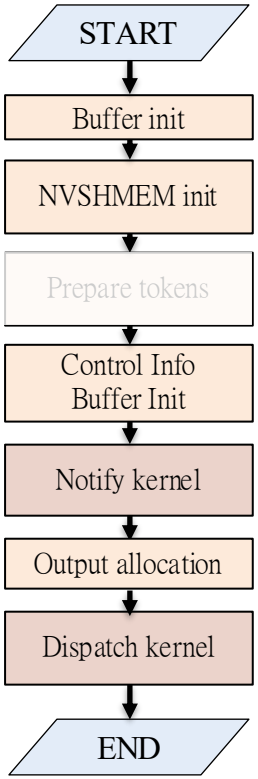


Same token → one remote node → several final GPU ranks

- RDMA connects the same local GPU index across nodes; NVLink reaches the final local ranks.
- A token crosses RDMA once per destination node, then its rank mask drives local forwarding.

High-Throughput Dispatch

Notify and Dispatch Kernels



DeepEP v1.2.1 · Reference R24

Notify: establish the layout

Exchange token counts.
Compute receive prefixes and output sizes.

Dispatch: move the tokens

Use the prepared layout to pipeline RDMA and NVLink transfers.

· Notify exchanges counts and builds prefixes; the runtime then allocates the receive tensors.
· Dispatch uses that metadata to move payloads through the RDMA and NVLink pipeline.

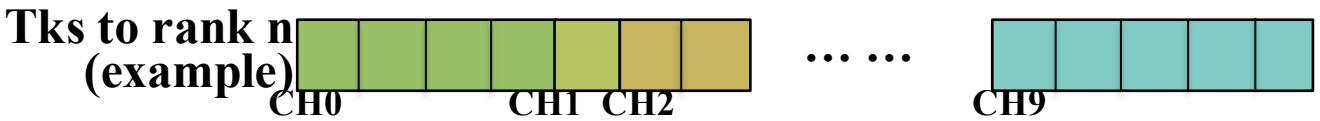
High-Throughput Dispatch

Communication Channels

Partition the source-token index range



Select the subset destined for one peer

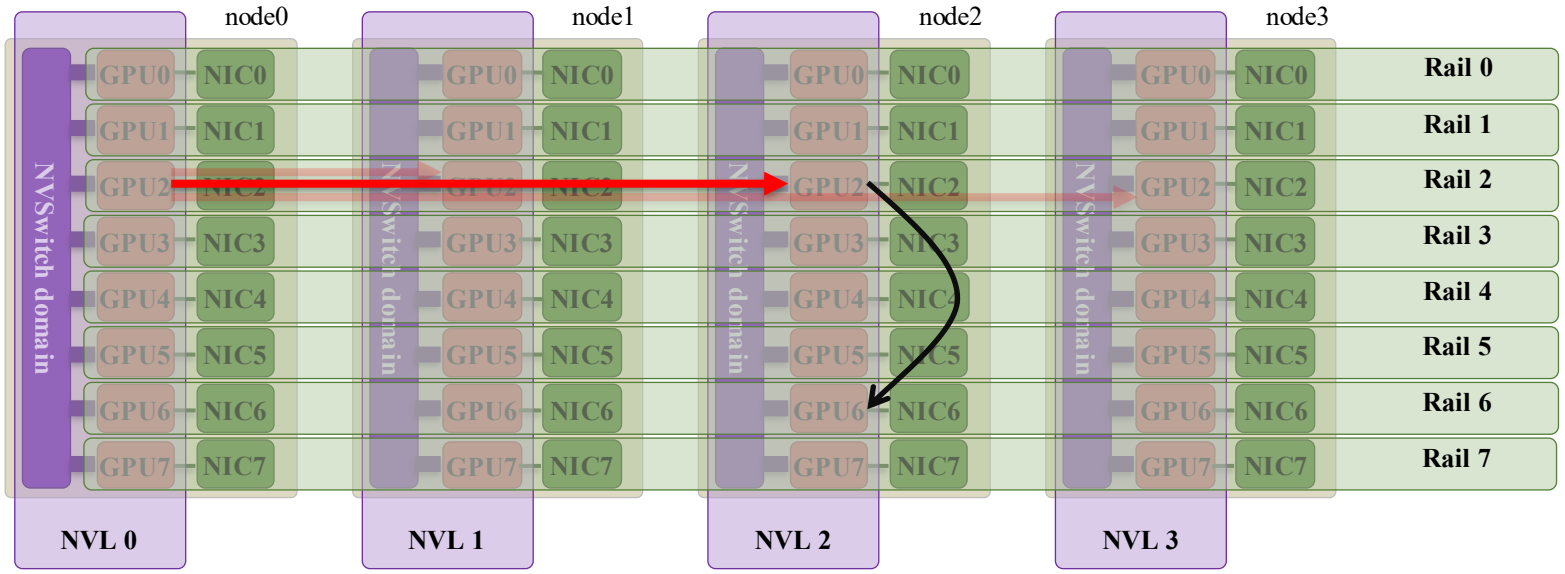


A channel is a software pipeline partition. A QP is transport state.

· A channel partitions input token indices and owns independent communication progress.
· C_h = 10 launches 20 CTAs: one sender/receiver CTA and one forwarder CTA per channel.

High-Throughput Dispatch

Exchanging Token Counts



1 Exchange per-node / per-rank counts over RDMA

2 Gather the final-rank / expert counts within the NVLink domain

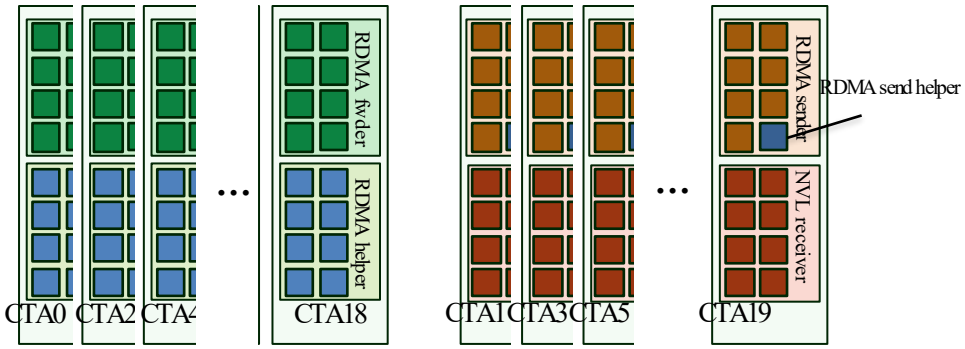
DeepEP v1.2.1 · Reference R26, R29, R30, R31

· Node counts deduplicate each token within a destination node; rank and expert counts stay distinct.

· Counts follow the payload route: exchange across an RDMA rail, then distribute through NVLink.

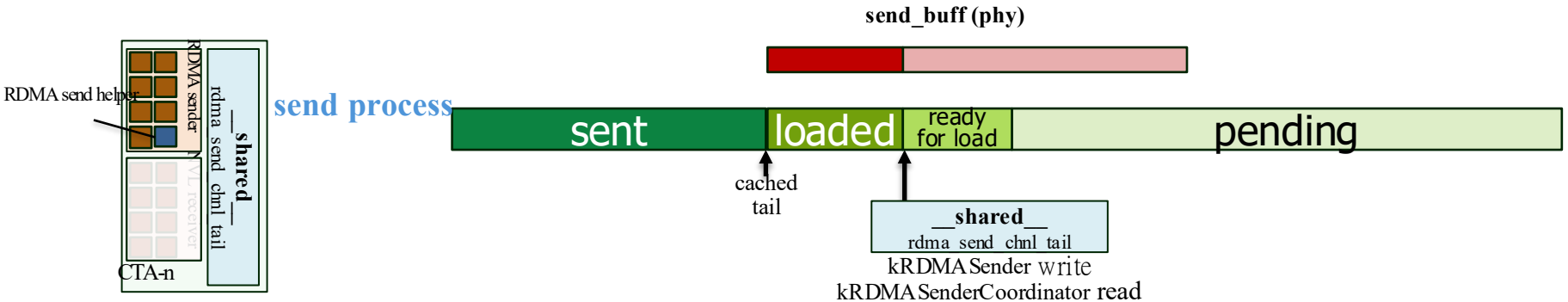
High-Throughput Dispatch

Sender and Coordinator



Sender CTA: load token chunks; publish progress to the coordinator.

Forwarder CTA: 1 coordinator warp is active; 7 warps retire.

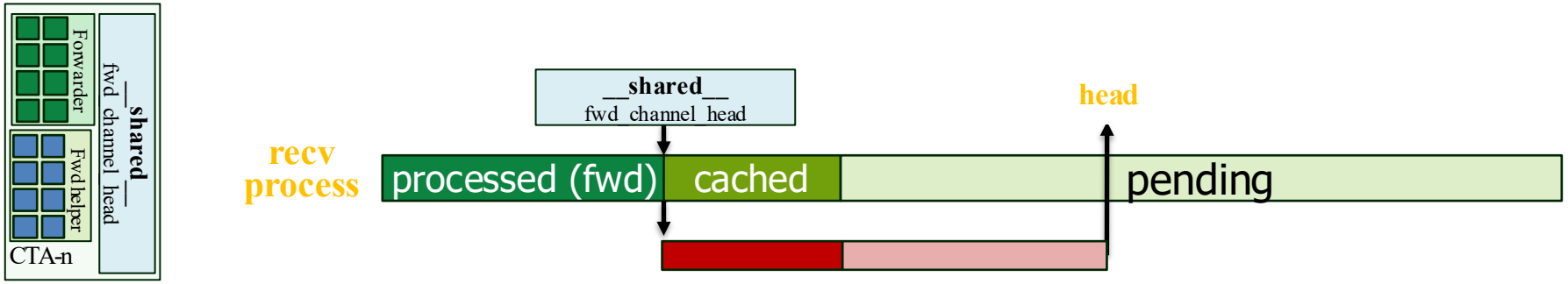


- Sender warps pack token data and routing metadata, then publish contiguous ready progress.
- A coordinator in the same CTA observes shared progress and issues chunked RDMA writes.

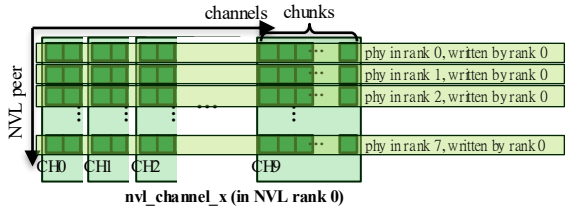
High-Throughput Dispatch

Forwarder and NVLink Receiver

RDMA receive → forwarding progress



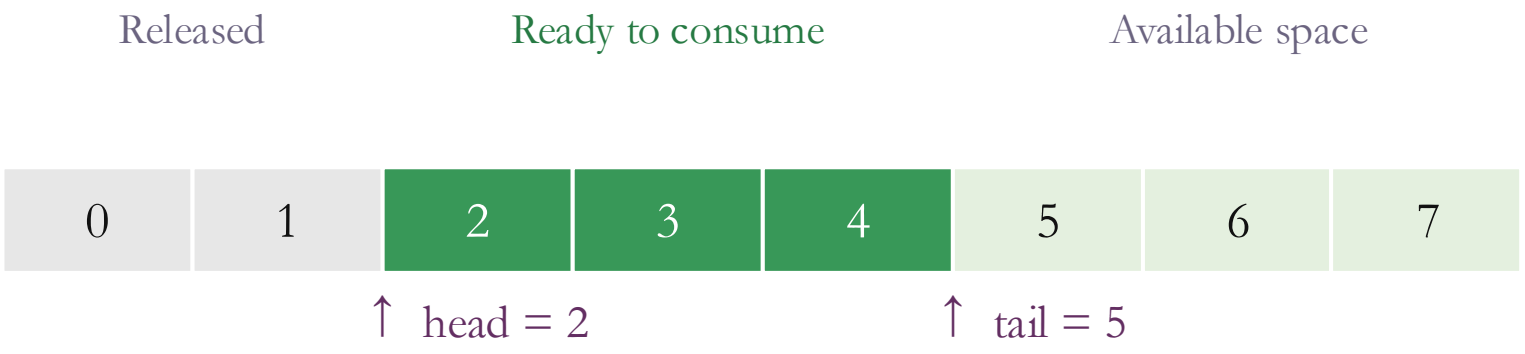
NVLink receiver consumes its local buffer



- Forwarder warps inspect destination bits and copy arrived tokens into each local rank's NVLink ring.
- NVLink receiver warps copy ready tokens to final offsets and release consumed ring slots.

High-Throughput Dispatch

Buffer Ownership and Reuse



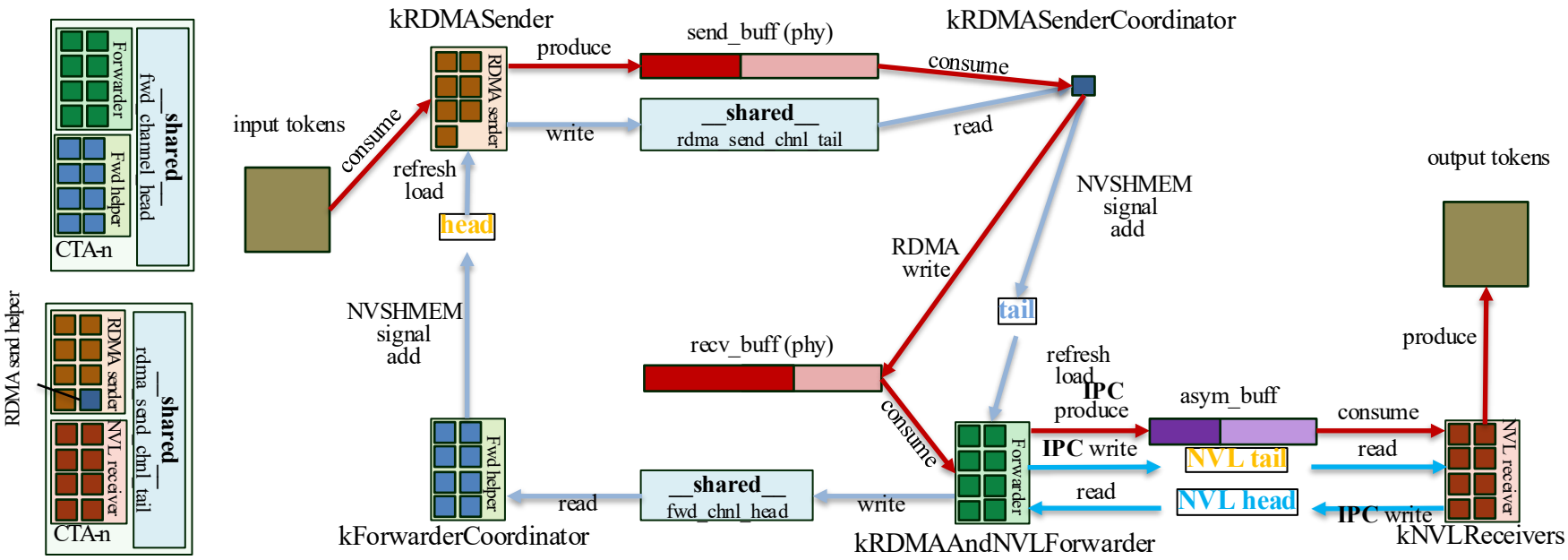
Location	Writer	Reader
Receiver data / tail	Forwarder	NVLink receiver
Forwarder head	NVLink receiver via IPC	Forwarder

DeepEP v1.2.1 · Reference R36, R41

· RDMA stages through separate send/receive regions; NVLink writes into the receiver's data ring.
· Reuse follows consumer release: remote forwarders release RDMA slots; final receivers release NVLink slots.

High-Throughput Dispatch

The Complete Communication Pipeline



· Input -> sender packing -> RDMA ring -> NVLink forwarding -> compact receive tensor.
· Each stage publishes readiness downstream and returns reusable capacity upstream.

DeepEP Source Map

Python APIs and GPU Kernels

Python API	`deep_ep/buffer.py`
↓	get_dispatch_layout / dispatch / low_latency_dispatch
Native runtime	`csrc/deep_ep.cpp`
↓	Resource setup and kernel selection
GPU kernels	`layout.cu; internode.cu; internode_ll.cu`
↓	Layout, staging, forwarding and packing
Device transport	`ibgda_device.cuh`
	WQE construction and NIC submission

DeepEP v1.2.1

- Buffer.get_dispatch_layout() builds route counts; Buffer.dispatch() selects the communication path.
- C++ manages tensors and launches; CUDA kernels implement notify, dispatch, and queue progress.